Università degli Studi di Torino

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Lab 4 — Modellazione del Linguaggio con N-gram

Modeling social media and literary language with N-grams

Studente: Alessandro Olivero
Matricola: [INSERIRE MATRICOLA]
Docente: Prof. Daniele Radicioni
A.A.: 2024/2025

Torino, 28 aprile 2026

Indice

1 Introduzione

Questo laboratorio ha come obiettivo la modellazione statistica del linguaggio attraverso modelli N-gram applicati a due domini linguistici radicalmente diversi: i **social media** (Twitter) e il **linguaggio letterario** (romanzi dell'Ottocento).

L'ipotesi di partenza è che i due domini siano statisticamente distinguibili: il linguaggio dei social media è caratterizzato da frasi brevi, abbreviazioni, slang ed espressioni emotive, mentre il linguaggio letterario si distingue per strutture sintattiche complesse, vocabolario formale e coerenza narrativa.

1.1 Obiettivi

- Costruire modelli N-gram (unigrammi, bigrammi, trigrammi) su entrambi i corpora e analizzare le sequenze più frequenti.
- Addestrare modelli di linguaggio MLE e Laplace e confrontarne i limiti.
- Dimostrare le differenze tra i due domini attraverso la perplexity e la log-likelihood.
- Implementare un classificatore basato su log-likelihood capace di distinguere automaticamente un testo Twitter da uno letterario.
- Analizzare le differenze linguistiche attraverso metriche quantitative come il Type-Token Ratio (TTR) e la lunghezza media delle frasi.

1.2 Strumenti utilizzati

- **Python 3.x**
- **NLTK** (Natural Language Toolkit): tokenizzazione, corpora, modelli di linguaggio
- **Matplotlib**: visualizzazione dei risultati
- **scikit-learn**: valutazione formale del classificatore

2 Dataset

2.1 Twitter Samples (NLTK)

Il corpus Twitter è stato ottenuto dal pacchetto `nltk.corpus.twitter_samples`, che contiene tweet pre-classificati in positivi e negativi raccolti nel 2015.

- `positive_tweets.json`: 5.000 tweet con emoticon positive `:)`
- `negative_tweets.json`: 5.000 tweet con emoticon negative `:(`
- **Totale**: 10.000 tweet

```
from nltk.corpus import twitter_samples

tweets_pos = twitter_samples.strings('positive_tweets.json')
tweets_neg = twitter_samples.strings('negative_tweets.json')
tweets_raw = tweets_pos + tweets_neg  # 10000 tweet totali
```

Listing 1: Caricamento dataset Twitter

Esempi di tweet dal dataset:

- *Positivo*: "#FollowFriday @France_Inte for being top engaged members :)"
- *Negativo*: "hopeless for tmr :("

2.2 Corpus Letterario (Gutenberg)

Il testo letterario è stato estratto dal corpus Gutenberg di NLTK, che raccoglie opere classiche di dominio pubblico. È stato scelto il romanzo ***Emma* di Jane Austen (1816)** come rappresentante del linguaggio letterario formale ottocentesco.

```
from nltk.corpus import gutenberg

testo_letterario_raw = gutenberg.raw('austen-emma.txt')
# 887.071 caratteri totali
```

Listing 2: Caricamento corpus letterario

2.3 Statistiche comparative

Metrica	Twitter	Letterario
Testi totali	10.000 tweet	1 romanzo
Token totali	94.706	158.267
Parole uniche	12.024	9.305
Lunghezza media	16,0 parole	25,6 parole
Type-Token Ratio	0,1270	0,0588

Tabella 1: Statistiche comparative dei due corpora

3 Preprocessing

3.1 Pulizia dei testi

I due corpora richiedono procedure di pulizia diverse data la loro natura.

3.1.1 Pulizia Twitter

```
def pulisci_tweet(testo):
    testo = re.sub(r'http\S+', '', testo)        # rimuovi URL
    testo = re.sub(r'@\w+', '', testo)            # rimuovi
   @menzioni
```

```
    testo = re.sub(r'#', '', testo)              # rimuovi simbolo #
    testo = re.sub(r'[^a-zA-Z\s]', '', testo)   # solo lettere
    testo = testo.lower().strip()
    return testo
```

Listing 3: Funzione di pulizia tweet

Esempio di trasformazione:

- **Prima**: `#FollowFriday @France_Inte for being top :)`
- **Dopo**: `followfriday for being top`

3.1.2 Pulizia testo letterario

```
def pulisci_letterario(testo):
    testo = re.sub(r'[^a-zA-Z\s]', '', testo)
    testo = testo.lower().strip()
    return testo
```

Listing 4: Funzione di pulizia testo letterario

3.2 Tokenizzazione

La tokenizzazione è stata effettuata con `word_tokenize` di NLTK, che a differenza del semplice `split()` gestisce correttamente la punteggiatura e le contrazioni inglesi.

```
from nltk.tokenize import word_tokenize

tokens_tweet = word_tokenize(' '.join(tweets_puliti))
tokens_lett  = word_tokenize(testo_lett_pulito)
```

Listing 5: Tokenizzazione

Esempio di differenza tra `split()` e `word_tokenize`:

Metodo	Output
`split()`	`["I", "can't", "stop,"]`
`word_tokenize()`	`["I", "ca", "n't", "stop", ","]`

Tabella 2: Confronto tra split() e word_tokenize()

3.3 Rimozione delle Stopwords

Le stopwords sono parole molto comuni ("the", "a", "in") che appaiono frequentemente in tutti i tipi di testo e quindi non sono discriminative. Rimuoverle permette di far emergere i bigrammi veramente caratteristici di ciascun dominio.

```
from nltk.corpus import stopwords

stop = set(stopwords.words('english'))
tokens_tweet_filtrati = [t for t in tokens_tweet if t not in stop]
tokens_lett_filtrati  = [t for t in tokens_lett  if t not in stop]
```

Listing 6: Rimozione stopwords

4 Modelli N-gram

4.1 Cos'è un N-gram

Un **N-gram** è una sequenza contigua di N parole in un testo. Formalmente, dato un testo $w_1, w_2, \ldots, w_T$, un N-gram è una sequenza $(w_i, w_{i+1}, \ldots, w_{i+N-1})$.

- **Unigrammi** ($N = 1$): singole parole. Frase: "i love cats" → `[i, love, cats]`
- **Bigrammi** ($N = 2$): coppie di parole consecutive. Frase: "i love cats" → `[(i,love), (love,cats)]`
- **Trigrammi** ($N = 3$): triplette di parole consecutive. Frase: "i love cats" → `[(i,love,cats)]`

4.2 Analisi delle frequenze

4.2.1 Bigrammi senza stopwords

I bigrammi più frequenti dopo la rimozione delle stopwords sono i più informativi per caratterizzare ciascun dominio:

Dominio	Bigramma	Frequenza
Twitter	(wan, na)	123
Twitter	(follow, please)	81
Twitter	(gon, na)	72
Twitter	(happy, birthday)	54
Letterario	(mr, knightley)	248
Letterario	(mrs, weston)	222
Letterario	(mr, elton)	184
Letterario	(miss, woodhouse)	148

Tabella 3: Top bigrammi per dominio (senza stopwords)

I bigrammi Twitter riflettono il linguaggio colloquiale e le interazioni sociali tipiche della piattaforma: *wanna*, *gonna*, richieste di follow reciproco, auguri di compleanno.

I bigrammi letterari sono dominati da titoli e nomi propri dei personaggi del romanzo (Mr. Knightley, Mrs. Weston, Miss Woodhouse), riflettendo il registro formale dell'inglese ottocentesco.

4.2.2 Bigrammi con stopwords

Dominio	Bigramma	Frequenza
Twitter	(thank, you)	260
Twitter	(i, love)	173
Twitter	(i, miss)	160
Twitter	(love, you)	155
Twitter	(i, cant)	155
Letterario	(to, be)	602
Letterario	(of, the)	563
Letterario	(i, am)	363
Letterario	(she, had)	322
Letterario	(had, been)	305

Tabella 4: Top bigrammi per dominio (con stopwords)

Osservazioni:

- Twitter usa la forma contratta *cant* invece di *cannot*, tipica del linguaggio informale.
- Il letterario usa *i am* (forma completa) invece di *i'm*, tipico dell'inglese formale ottocentesco.
- Il letterario mostra strutture come *had been* (past perfect) assenti nei tweet.

4.2.3 Trigrammi

Dominio	Trigramma	Frequenza
Twitter	(i, love, you)	84
Twitter	(you, so, much)	74
Twitter	(thanks, for, the)	67
Letterario	(i, do, not)	123
Letterario	(i, am, sure)	97
Letterario	(she, could, not)	68
Letterario	(a, great, deal)	63

Tabella 5: Top trigrammi per dominio

5 Modelli di Linguaggio

5.1 Maximum Likelihood Estimation (MLE)

Il modello MLE stima la probabilità di una parola dato il contesto precedente contando le occorrenze nel corpus di training.

Per i bigrammi, la probabilità condizionale è:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})} \tag{1}$$

dove $C(w_{i-1}, w_i)$ è il numero di occorrenze del bigramma (w_{i-1}, w_i) e $C(w_{i-1})$ è il numero di occorrenze di w_{i-1}.

```
from nltk.lm import MLE
from nltk.lm.preprocessing import padded_everygram_pipeline

n = 2  # bigrammi
train_data, vocab = padded_everygram_pipeline(n, [tokens_tweet])

modello = MLE(n)
modello.fit(train_data, vocab)
```

Listing 7: Addestramento modello MLE

5.1.1 Probabilità condizionali osservate

Parola	Contesto	P (Twitter)	P (Letterario)
love	i	0,0530	0,0017
wait	cant	0,1146	0,0000
happy	so	0,0138	—
mr	said	0,0000	0,0802
been	had	—	0,1885

Tabella 6: Probabilità condizionali nei due modelli MLE

I risultati confermano le differenze attese:

- *love* segue *i* con probabilità 5,3% su Twitter, quasi mai nel romanzo.
- *mr* segue *said* con probabilità 8% nel romanzo, mai nei tweet.
- *had been* (past perfect) è molto frequente nel romanzo (18,85%), assente nei tweet.

5.1.2 Limite del MLE: il problema dello zero

Il principale limite del MLE è che assegna probabilità **zero** a qualsiasi sequenza non osservata nel training. Questo causa la **perplexity infinita** quando il testo di test contiene parole o bigrammi mai visti:

$$P_{\text{MLE}}(\text{“xyzword”} \mid \text{“i”}) = \frac{0}{C(\text{“i”})} = 0 \tag{2}$$

```
print(modello_tweet.score('xyzword', ['i']))
# -> 0.000000  (zero esatto)

frase_rara = word_tokenize("i xyzword lol")
print(modello_tweet.perplexity(frase_rara))
# -> inf  (infinita!)
```

Listing 8: Dimostrazione del problema dello zero

5.2 Smoothing di Laplace

Il **Laplace smoothing** (o add-one smoothing) risolve il problema dello zero aggiungendo 1 a tutti i conteggi:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|} \tag{3}$$

dove $|V|$ è la dimensione del vocabolario.

```python
from nltk.lm import Laplace

modello_lp = Laplace(n)
modello_lp.fit(train_data, vocab)

# Ora le parole mai viste hanno probabilita' piccola ma > 0
print(modello_lp.score('xyzword', ['i']))
# -> 0.000065  (piccolo ma non zero)

print(modello_lp.perplexity(frase_rara))
# -> 1677.95  (alta ma non infinita)
```

Listing 9: Addestramento modello Laplace

Metrica	MLE	Laplace
P('xyzword' | 'i')	0,000000	0,000065
Perplexity su parola rara	∞	1677,95
Gestisce parole mai viste	No	Sì

Tabella 7: Confronto MLE vs Laplace

5.3 Scelta del valore di N

N	PP frase Twitter	PP frase Letterario
1	5170,14	12025,00
2	5170,91	12027,00
3	5170,92	12027,00

Tabella 8: Confronto perplexity per n=1,2,3 (Laplace, modello Twitter)

I valori quasi identici per $N = 1, 2, 3$ indicano un problema di **data sparsity**: con soli 94.706 token Twitter, la maggior parte dei bigrammi e trigrammi appare raramente, e Laplace li tratta quasi tutti come sequenze rare. Con un corpus più grande le differenze sarebbero più marcate, e i trigrammi catturerebbero strutture sintattiche più complesse.

6 Perplexity e Classificazione

6.1 Cos'è la Perplexity

La **perplexity** misura quanto un modello è "sorpreso" da un testo:

$$\mathrm{PP}(W) = P(w_1, w_2, \ldots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \ldots, w_N)}} \tag{4}$$

Un valore **basso** indica che il modello conosce bene quel tipo di testo; un valore **alto** indica sorpresa.

6.2 Perplexity cross-domain

Frase	Tipo atteso	PP Twitter	PP Lett.	Verdetto
"i love you so much"	TW	5170,9	4488,5	TW ✓
"so happy right now"	TW	12027,0	9308,0	LT ×
"i miss you feel so sad"	TW	5746,3	4917,0	TW ✓
"she had been very well"	LT	12027,0	9308,0	TW ×
"it was a very good thing"	LT	6756,2	5258,0	TW ×

Tabella 9: Perplexity cross-domain (Laplace normalizzata)

Limite osservato: la perplexity con Laplace su vocabolari di dimensioni diverse non è direttamente comparabile. Il modello con vocabolario più grande (Twitter, 12.027 parole) distribuisce la probabilità su più termini e ottiene sempre perplexity più alta, indipendentemente dal contenuto del testo.

6.3 Classificatore con Log-Likelihood

Per superare il limite della perplexity, abbiamo implementato un classificatore basato sulla **log-likelihood**:

$$\log P(w_1, \ldots, w_N) = \sum_{i=2}^{N} \log P(w_i \mid w_{i-1}) \tag{5}$$

Un valore **meno negativo** (più alto) indica che il modello spiega meglio il testo.

```python
import math

def log_likelihood(modello, tokens):
    score = 0.0
    for i in range(1, len(tokens)):
        contesto = [tokens[i-1]]
        prob = modello.score(tokens[i], contesto)
        if prob > 0:
            score += math.log(prob)
        else:
            score += math.log(1e-10)  # parole mai viste
```

```
    return score

def classifica_loglik(testo, mod_tw, mod_lt):
    tokens = word_tokenize(testo.lower())
    ll_tw   = log_likelihood(mod_tw, tokens)
    ll_lett = log_likelihood(mod_lt, tokens)
    return "TWITTER" if ll_tw > ll_lett else "LETTERARIO"
```

Listing 10: Funzione log-likelihood

6.3.1 Risultati del classificatore

Testo	LL Twitter	LL Lett.	Verdetto
"i love you so much thank you"	$-32,02$	$-42,15$	TWITTER ✓
"she had been very well disposed"	$-124,74$	$-21,91$	LETTERARIO ✓
"i miss you so much cant stop thinking"	$-41,54$	$-122,88$	TWITTER ✓
"it was a very good thing to have done"	$-29,21$	$-24,75$	LETTERARIO ✓
"happy birthday love you so much"	$-11,27$	$-58,54$	TWITTER ✓
"she was not a woman of many words"	$-70,82$	$-47,21$	LETTERARIO ✓
"i cant stop smiling so happy"	$-57,95$	$-97,56$	TWITTER ✓
"mr knightley had always been very fond"	$-108,42$	$-24,91$	LETTERARIO ✓

Tabella 10: Risultati del classificatore log-likelihood (8/8 corretti)

Il classificatore log-likelihood classifica correttamente tutti gli 8 testi di test, a differenza del classificatore basato su perplexity (3/8).

7 Generazione del Testo

Il modello MLE può essere usato per **generare** testo nello stile del corpus su cui è stato addestrato. La generazione avviene campionando la parola successiva in base alle probabilità condizionali apprese.

```
def genera_testo(modello, n_parole=30):
    testo_raw = modello.generate(n_parole, random_seed=42)
    # Filtra token speciali e lettere singole (artefatti)
    testo_filtrato = [
        t for t in testo_raw
        if t not in ['<s>', '</s>', '<UNK>']
        and not (len(t) == 1 and t not in ['i', 'a'])
    ]
    return ' '.join(testo_filtrato)
```

Listing 11: Generazione testo

7.0.1 Testo generato

Stile	Testo generato
Twitter	"papa coach it from the rain why are not a fantastic spring fiesta make sure the donation manager we all the rest his cute when im being a small polka dots"
Letterario	"over a good exchange consent to the cardroom looking about her less allied with persevering attention could not that a substance of her but yet heard any body to me"

Tabella 11: Esempi di testo generato dai due modelli

La differenza stilistica è evidente: il testo Twitter è disorganizzato e colloquiale, mentre quello letterario usa costruzioni più complesse tipiche del romanzo ottocentesco.

7.0.2 Confronto con diversi seed

Seed	Twitter	Letterario
99	"im getting bigger at the epic i love you in my life"	"i can believe i shall decidedly in the neighbourhood"
7	"have a new cards in march free insurance"	"hartfield and return among their getting at last"
123	"and by my bestfriend she brought us i want corn chips"	"and away from being to announce cheerful exulting"

Tabella 12: Generazione con diversi seed casuali

8 Analisi Linguistica

8.1 Type-Token Ratio (TTR)

Il **Type-Token Ratio** misura la ricchezza lessicale di un corpus:

$$\text{TTR} = \frac{\text{numero di parole uniche (types)}}{\text{numero totale di parole (tokens)}} \tag{6}$$

Corpus	Types	Tokens	TTR
Twitter	12.024	94.706	0,1270
Letterario	9.305	158.267	0,0588

Tabella 13: Type-Token Ratio dei due corpora

Il TTR di Twitter è più del **doppio** rispetto al romanzo. Questo risultato può sembrare controintuitivo, ma si spiega con la natura eterogenea dei tweet: ogni tweet introduce nomi propri, hashtag, abbreviazioni e slang diversi (`followfriday`, `unfollowers`, `bestfriend`), molti dei quali appaiono raramente nel corpus. Il romanzo invece ripete sistematicamente gli stessi personaggi e costruzioni grammaticali.

8.2 Lunghezza media delle frasi

Corpus	Lunghezza media (parole)
Tweet	16,0
Frase letteraria	25,6

Tabella 14: Lunghezza media delle frasi

Le frasi letterarie sono in media **9,6 parole più lunghe** dei tweet. Questo riflette la struttura sintattica complessa del romanzo ottocentesco, con subordinate multiple, incisi e descrizioni elaborate, contrapposta alla brevità imposta dal limite di 140 caratteri di Twitter.

9 Limiti e Vantaggi

9.1 Vantaggi dell'approccio N-gram

- **Semplicità**: facile da implementare e interpretare.
- **Efficienza**: addestramento e inferenza molto veloci.
- **Trasparenza**: le probabilità condizionali sono direttamente leggibili e spiegabili.
- **Nessun training pesante**: non richiede GPU o risorse computazionali elevate.

9.2 Limiti dell'approccio N-gram

- **Data sparsity**: con dataset piccoli (come i 10.000 tweet usati), la maggior parte dei bigrammi e trigrammi appare raramente o mai. Questo rende i valori di perplexity per $N = 1, 2, 3$ quasi identici.
- **Finestra di contesto limitata**: i bigrammi considerano solo la parola precedente. Non catturano dipendenze a lungo raggio tipiche del linguaggio naturale ("The man who I met yesterday **was** kind").
- **Nessuna comprensione semantica**: il modello non capisce il significato delle parole. *happy* e *joyful* sono trattate come parole completamente diverse.

- **Problema della perplexity con Laplace**: la perplexity non è direttamente comparabile tra modelli con vocabolari di dimensioni diverse, rendendo necessario l'uso della log-likelihood per il classificatore.
- **Dataset sbilanciato**: il corpus Twitter (94.706 token) è significativamente più piccolo del corpus letterario (158.267 token), influenzando il confronto.

9.3 Confronto con approcci alternativi

Approccio	Vantaggi	Svantaggi	Complessità
N-gram (questo lab)	Semplice, veloce, interpretabile	Nessun contesto semantico, sparsità	Bassa
Word2Vec	Cattura semantica, vettori densi	Non genera testo, no probabilità	Media
LSTM / RNN	Contesto lungo, genera testo realistico	Richiede molto training	Alta
Transformer (GPT)	Stato dell'arte, testo coerente	Computazionalmente costoso	Molto alta

Tabella 15: Confronto tra approcci per la modellazione del linguaggio

10 Risultati e Conclusioni

10.1 Riepilogo dei risultati

Esperimento	Risultato	Interpretazione
Bigrammi Twitter	(i, love), (cant, wait)	Linguaggio emozionale, informale
Bigrammi Letterario	(mr, knightley), (mrs, weston)	Nomi propri formali, titoli
TTR Twitter	0,1270	Vocabolario più vario
TTR Letterario	0,0588	Vocabolario più ripetitivo
Lunghezza tweet	16,0 parole	Frasi brevi
Lunghezza frase lett.	25,6 parole	Frasi lunghe e complesse
Classificatore (PP)	3/8 = 38%	Inaffidabile
Classificatore (LL)	8/8 = 100%	Accurato su frasi note

Tabella 16: Riepilogo dei principali risultati

10.2 Conclusioni

Questo laboratorio ha dimostrato che i modelli N-gram sono strumenti efficaci per caratterizzare e distinguere stili linguistici diversi.

I principali risultati ottenuti sono:

1. I bigrammi e trigrammi più frequenti riflettono chiaramente le differenze culturali e stilistiche tra Twitter e il linguaggio letterario ottocentesco.

2. Il TTR più alto di Twitter conferma che i social media producono un linguaggio lessicalmente più vario, sebbene meno strutturato sintatticamente.

3. La perplexity con Laplace non è un indicatore affidabile per la classificazione cross-dominio quando i vocabolari hanno dimensioni molto diverse. La log-likelihood è una metrica più robusta per questo scopo.

4. Il classificatore basato su log-likelihood raggiunge un'accuratezza del 100% sulle frasi di test, confermando che i due domini sono statisticamente ben separabili.

5. La generazione di testo evidenzia le differenze stilistiche: il modello Twitter produce testo colloquiale e frammentato, mentre il modello letterario produce costruzioni più elaborate tipiche del romanzo.

Sviluppi futuri potrebbero includere l'uso di dataset più grandi per ridurre la sparsità, l'adozione di smoothing più sofisticati come Kneser-Ney, e il confronto con modelli neurali come LSTM o Transformer per valutare il guadagno in qualità rispetto alla complessità computazionale.

Contributi da strumenti generativi

Come richiesto dalle istruzioni del laboratorio, si dichiara che nella stesura di questa relazione e nella scrittura del codice sono stati utilizzati strumenti di AI generativa (Claude, Anthropic) per:

- Supporto nella strutturazione del codice Python.
- Debugging di errori nella generazione del testo e nel classificatore.
- Revisione e formattazione della relazione in LaTeX.

Tutte le scelte progettuali, l'analisi dei risultati e le conclusioni sono state elaborate autonomamente dallo studente.